

DeGro: Target Determinacy Verification and Grounded Repair for Executable Mathematical Reasoning

Anonymous Author(s)
Submission Id: ANONYMOUS

Abstract

Executable mathematical reasoning may run successfully while solving an underconstrained or mistranslated problem. Execution checks expose syntax, runtime, and arithmetic failures but do not establish that the encoded constraints determine the requested quantity. We introduce *DeGro*, a neuro-symbolic framework for *target determinacy diagnosis*. DeGro asks a solver for two feasible assignments that disagree on the target; when they exist, a grounded policy adds only a source-supported constraint that restores determinacy or abstains. On a fixed paired evaluation that varies source support while holding the formalization constant, DeGro improves repair-or-abstain accuracy for GPT-OSS 20B and 120B. Human-verified natural-error studies show that DeGro improves accuracy on the supported MATH-500 cohort, while higher-coverage ALG514 reveals answer-compatible formalization errors. Detection is much easier than repair, and concrete witnesses add no measurable benefit beyond the verdict. Target determinacy is therefore an auditable intermediate guardrail, not a proof of source-to-formal faithfulness.

CCS Concepts

• **Computing methodologies** → **Knowledge representation and reasoning**; **Intelligent agents**; *Natural language processing*.

Keywords

neuro-symbolic reasoning, mathematical agents, verification, counterexamples, abstention

ACM Reference Format:

Anonymous Author(s). 2027. DeGro: Target Determinacy Verification and Grounded Repair for Executable Mathematical Reasoning. In *Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027)*, Hanoi, Vietnam, 3–7 May 2027, IFAAMAS, 10 pages.

1 Introduction

Large language models increasingly delegate mathematical computation to programs and symbolic solvers. Program-aided reasoning reduces arithmetic errors, while systems such as SymCode generate self-contained symbolic programs and repair execution failures [5, 15]. Successful execution, however, does not establish that the encoded constraints determine the requested answer. A program may run and return a value even when its formalization admits several values of the target.



This work is licensed under a Creative Commons Attribution International 4.0 License.

Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027), M. Baldoni, F. Fang, W. Yeoh, N. Yorke-Smith (eds.), 3–7 May 2027, Hanoi, Vietnam. © 2027 International Foundation for Autonomous Agents and Multiagent Systems (www.ifaamas.org).

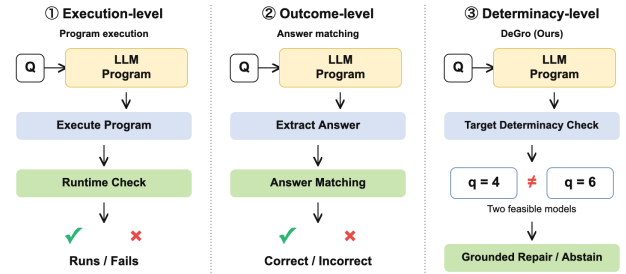


Figure 1: Three levels of executable mathematical reasoning evaluation. Execution checks whether a program runs; outcome evaluation checks its final answer. DeGro diagnoses target ambiguity through two feasible models with different target values (illustrated by $q = 4$ and $q = 6$), then attempts source-grounded repair with deterministic acceptance gates or abstains. Target determinacy is relative to the encoded specification and does not establish faithfulness to the source problem.

Consider: “Ten tasks are divided between A and B. A receives two more than B. How many does A receive?” A generated model containing only $a + b = 10$ is executable and satisfiable, but $(a, b) = (4, 6)$ and $(6, 4)$ imply different values of the target a . We call this condition *target ambiguity*: feasible assignments disagree on the requested quantity. It concerns the target rather than uniqueness of the entire assignment, since freedom in other variables need not affect the answer.

Detecting ambiguity does not establish how to resolve it. The same model faithfully represents the shorter source “Ten tasks are divided between A and B. How many does A receive?” For the complete source, adding $a = b + 2$ recovers an omitted relation; for the shorter source, the same addition fabricates information. The formalization and solver verdict are identical, but the correct action changes with source support. A repair policy must therefore distinguish evidence that the target is ambiguous from evidence that a particular constraint is justified.

We propose DeGro, a neuro-symbolic framework that separates these two decisions. It first constructs two renamed copies of a generated specification and asks whether both can satisfy the constraints while disagreeing on the target. A satisfying query provides auditable evidence of ambiguity. The agent then proposes a constraint with supporting source text, and acceptance checks require consistency, non-redundancy, and restored target determinacy. If no admissible repair is found, the agent abstains. The solver guarantee concerns the encoded specification: even a determinate target can be wrong when the target or constraints were mistranslated.

We evaluate this framework at two complementary levels. A controlled paired benchmark holds the formalization and diagnostic fixed while varying source support, testing whether the policy repairs a supported omission and abstains on its underspecified counterpart. Human-verified natural-error studies then examine representation coverage, ambiguity detection, and successful grounded repair separately. The controlled results show improved repair-or-abstain decisions, while the natural errors expose a substantial gap between detecting ambiguity and repairing its cause. We also test whether concrete witness assignments improve decisions beyond the solver verdict.

Our research questions are:

(RQ1) Decision quality and tradeoffs. How do target-determinacy diagnosis and source grounding affect repair-or-abstain accuracy and the tradeoff between supported repair and unsupported additions?

(RQ2) Paired source sensitivity. When the formalization and diagnostic are held fixed, can an agent both repair the source-supported omission and abstain on the underspecified member of the same pair?

(RQ3) Practical bottlenecks. How do representation coverage, ambiguity detection, and grounded repair limit effectiveness on natural errors, and do concrete witnesses improve controlled repair decisions beyond the verdict?

Takeaway

DeGro checks whether the encoded constraints determine the requested target, accepts only source-supported repairs, and otherwise abstains.

Our main contributions are:

(1) A target-determinacy-guided repair-or-abstain framework. We introduce DeGro, which combines two-copy target-determinacy diagnosis with source-grounded repair, deterministic acceptance gates, and explicit abstention (RQ1).

(2) A controlled paired benchmark for grounded repair decisions. We construct matched cases that hold the formalization fixed while varying whether the source supports the missing constraint, isolating supported repair from unsupported completion (RQ2).

(3) A multi-stage empirical audit of executable reasoning. We combine controlled and human-verified natural-error evaluations to separate representation coverage, ambiguity detection, and grounded repair; we additionally test whether concrete witnesses improve decisions (RQ3).

The formal guarantee concerns F ; the empirical answers concern model and policy behavior.

2 Related Work

2.1 Executable Mathematical Reasoning

PAL and Program-of-Thoughts separate language-based reasoning from program execution to reduce computational errors [8, 15]. MathCoder and ToRA integrate code and tool calls into mathematical problem solving [18, 34], while faithful chain-of-thought connects generated rationales to executable symbolic procedures [25]. SymCode is particularly close to our setting: it represents mathematical reasoning as a SymPy program and uses execution feedback to repair programmatic failures [5]. These approaches motivate an additional check after successful execution. A generated artifact may return a value even though its constraints admit other values of the requested quantity. DeGro checks this property of the encoded feasible region and routes ambiguous specifications to grounded repair or abstention.

Outcome verification and process supervision assess candidate solutions or intermediate reasoning steps [10, 24, 32]. Self-Refine uses model-generated feedback, and CRITIC incorporates external tools into revision [17, 26]; evidence on intrinsic self-correction shows the limits of revision without reliable external feedback [20]. DeGro contributes a specific solver observation: two feasible assignments disagree on the target. This observation establishes ambiguity under the representation, independently of the predicted answer, but does not identify which source constraint should be added.

2.2 Symbolic Diagnosis and Relational Verification

Recent work addresses semantic failures that survive execution. ReLoop tests optimization formulations through solver-based parameter perturbations [22]. SymDiag translates reasoning steps into constraints and uses satisfiability and entailment checks to localize defects, with a Self-Auditor to distinguish translation errors from reasoning errors [11]. Minimal-core-guided repair instead returns an unsatisfiable subset of generated constraints as evidence for revising inconsistent translations [23, 29]. DeGro focuses on a complementary condition: the specification is satisfiable, but its feasible assignments disagree on the requested target. Its diagnosis concerns the specification as a whole rather than the validity of individual derivation steps, and its additive repair policy requires support from the original source.

The two-copy construction builds on self-composition and product programs, which reduce relational verification to queries over renamed copies [6, 7, 31]. DeGro instantiates this established technique as target agreement across feasible assignments. Agreement on the target is weaker than uniqueness of the full assignment: nuisance variables may remain free without making the answer ambiguous. Counterexample-guided refinement and synthesis likewise use solver evidence to guide changes [9, 30]. Here, however, a counterexample refutes target determinacy without supplying a semantically justified edit. Our contribution is the integration of this diagnosis with a source-sensitive repair policy and its evaluation, rather than a new relational verification primitive.

2.3 Grounded Repair, Abstention, and Faithfulness

Classical diagnosis, abductive inference, and formal model repair provide foundations for identifying defects and modifying specifications to satisfy a desired property [3, 13, 28]. LLM-based repair extends this direction to declarative formal specifications [2]. Restoring determinacy alone is insufficient in our setting, since a guessed equality can force a unique target. DeGro therefore requests a source-supported constraint and checks consistency, non-redundancy, and restored determinacy before acceptance. These formal gates verify the effect of the edit; source support remains a separate semantic requirement.

Selective prediction studies when to withhold an answer under a risk–coverage tradeoff [14, 16], and semantic uncertainty estimates uncertainty in language-model outputs [21]. DeGro bases its repair-or-abstain decision on two distinct questions: whether the encoded target is determinate and whether the source supports a constraint that resolves ambiguity. MIRA-Math is directly relevant because it evaluates requesting and integrating necessary missing information [1]. We adapt its validated latent states into matched sources with identical formalizations but different support for repair, isolating the decision to recover an omitted relation or abstain from unsupported completion.

Autoformalization also separates formal validity from preservation of natural-language meaning [4, 35]. DeGro’s guarantee is similarly representation-relative: a determinate specification can still contain a wrong target or an unsupported constraint. Our paired and human-verified natural-error evaluations therefore distinguish determinacy diagnosis, grounded repair, and answer accuracy rather than treating a solver verdict as proof of source-to-formal faithfulness.

3 Methodology

DeGro is a two-stage framework: it first diagnoses whether a generated specification determines the requested target, then converts an ambiguous verdict into either a source-grounded repair or a principled abstention. Figure 2 summarizes the complete loop.

3.1 Problem Setup, Determinacy, and Soundness

Let a natural-language problem be P . A model produces $F = (V, D, C, q)$, where V are variables, D their domains, C constraints, and $q(V)$ the requested expression. Define

$$\text{Sol}(F) = \{m \mid m \models D \wedge C\}.$$

The target is *determinate under F* iff

$$\forall m_1, m_2 \in \text{Sol}(F), \quad q(m_1) = q(m_2).$$

This is weaker than solution uniqueness: $x = 5, y \geq 0$ has many models, but target $q = x$ is determinate.

Construct renamed copies $F^{(1)}$ and $F^{(2)}$ and query

$$\Phi(F, q) = F^{(1)} \wedge F^{(2)} \wedge q(V^{(1)}) \neq q(V^{(2)}). \quad (1)$$

If (1) is satisfiable, its two projected models form a *target-divergent witness pair*. If it is unsatisfiable and F itself is satisfiable, the target is determinate.

Guarantee. For a sound solver and supported theory, $\text{SAT}(F)$ and an UNSAT result for Eq. (1) imply that all models of F agree on

q ; any disagreeing pair would satisfy the query. Timeout and UNKNOWN prove nothing. The guarantee is query-relative and strictly about F , not semantic equivalence $F \equiv P$: extra or wrong constraints may force a unique but mistranslated answer. Conversely, arbitrary strengthening can always force determinacy, motivating a separate source-grounding requirement. The policy therefore adds constraints only to ambiguous specifications; deleting or revising a determinate but wrong representation is outside scope.

Target determinacy is deliberately weaker than model uniqueness. A scheduling model may leave many assignments free while fixing total cost, and a word problem may admit several assignments to nuisance variables while fixing the requested quantity. Rejecting such cases would confuse representational freedom with answer ambiguity. A witness pair is also independently checkable: substitute each assignment into $D \wedge C$ and compare its target value. Determinacy is monotone under consistent strengthening, but the converse is unsafe because an arbitrary fact such as $q = 6$ can always collapse the feasible target set.

Running example. Let $V = \{a, b\}$ be nonnegative integers, $C = (a + b = 10)$, and $q = a$. The duplicated query can return $(a_1, b_1) = (4, 6)$ and $(a_2, b_2) = (6, 4)$. If P says “A receives two more than B,” then $a = b + 2$ is supported, consistent, non-redundant, and sufficient, yielding $a = 6$. Without that sentence, the same equation is a fabricated assumption and the correct action is abstention. The repair model sees neither the hidden gold value nor the missing gold constraint.

Structured representation. The agent emits one typed MODEL SPEC with variable sorts and bounds, source-linked constraints, and a target. Deterministic compilers translate it to computational and verification backends, avoiding independently generated SymPy and SMT programs. The supported fragment covers integer, real, and Boolean variables, arithmetic equalities and inequalities, finite domains, and selected modular constraints; other syntax yields NOT-SUPPORTED.

3.2 Stage I: Target-Determinacy Diagnosis

DeGro first tests $\text{SAT}(F)$, returning INCONSISTENT or UNKNOWN when appropriate, and otherwise evaluates Eq. (1). Unsatisfiability permits an answer under F ; satisfiability returns two assignments with different targets and triggers repair. This stage is deterministic: the model proposes F but does not judge determinacy.

Algorithm 1 Target-determinacy verification

Require: Typed specification $F = (V, D, C, q)$
Ensure: Status and optional witness pair

```

1:  $r_0 \leftarrow \text{CHECKSAT}(D \wedge C)$ 
2: if  $r_0 = \text{UNSAT}$  return INCONSISTENT
3: if  $r_0 \neq \text{SAT}$  return UNKNOWN
4:  $\Phi \leftarrow F^{(1)} \wedge F^{(2)} \wedge (q^{(1)} \neq q^{(2)})$  using disjoint copies
5:  $(r_1, M) \leftarrow \text{CHECKSAT}(\Phi)$ 
6: if  $r_1 = \text{UNSAT}$  return DETERMINATE
7: if  $r_1 \neq \text{SAT}$  return UNKNOWN
8: return AMBIGUOUS, PROJECTWITNESSES( $M$ )
```

The record stores both solver verdicts, normalized target values, and complete witnesses when available. Only AMBIGUOUS licenses repair; unsupported theories and timeouts are never coerced into Boolean outcomes.

3.3 Stage II: Grounded Repair or Abstention

The repair model receives P, C, q and the witness pair, then returns `ADD-CONSTRAINT` with an exact supporting span or `ABSTAIN`. A candidate must compile, cite text in P (or a declared domain rule), preserve satisfiability, be non-redundant, and restore determinacy. DeGro uses at most two rounds and otherwise returns `UNDERSPECIFIED`. Exact-span provenance is auditable but does not prove that the cited text was translated correctly.

The witness is diagnostic rather than corrective: it identifies a disagreement but does not reveal which missing relation the source licenses. The language model proposes that relation, and the deterministic gate decides whether it can enter the trusted specification. If an accepted addition yields a new divergent pair instead of determinacy, the second and final round receives the updated diagnosis. A rejected proposal never partially mutates F . Thus the loop is bounded and rollback-free, unlike unconstrained self-refinement whose successive edits may obscure their evidential basis.

3.4 Trusted Implementation and Repair Semantics

The implementation compiles a restricted expression AST to Z3 [12], rejecting arbitrary calls, undeclared names, malformed comparisons, and unsupported operators. Independent variable maps prevent aliasing between copies. Grounding is a deterministic deployment gate, not a prompt instruction; raw proposals and gated outcomes are both retained. The trusted base is the parser, compiler, solver, and gate, while the language model remains untrusted.

Integer, real, and Boolean declarations are typed explicitly, and declared bounds compile into constraints. The computational and verification backends consume the same normalized `MODELSPEC`; independently generated Python and SMT artifacts could otherwise disagree while both appear valid. Every record stores the initial verdict, proposed action, cited span, rejection reason, final verdict, model configuration, and prompt hash. This audit trail separates model behavior from enforcement behavior.

Repair acceptance. Let $\text{span}(c)$ be the source substring cited for candidate c , and let $\mathcal{R}$ be a frozen set of allowed domain-semantics rules. The gate accepts c exactly when

$$\text{Accept}(c) \iff \text{Compiles}(c) \wedge \text{Grounded}(c, P, \mathcal{R}) \wedge \text{SAT}(F \wedge c) \wedge \neg(F \models c) \wedge \text{Det}(F \wedge c, q), \quad (2)$$

where grounding requires an exact source span or named rule in $\mathcal{R}$. The first accepted candidate is committed; otherwise F is preserved. Thus redundant restatements, unsupported target pins, and insufficient additions cannot receive repair credit, while a supported expression need not match the gold syntax.

Complexity and failure states. Encoding size is linear in $|F|$, although solver time depends on the theory. A determinate input needs no repair call; an ambiguous one incurs at most two calls plus rechecks. We preserve distinct states: `UNSUPPORTED` for representation limits, `UNKNOWN` for solver inconclusiveness, `INCONSISTENT` for an unsatisfiable F , and `UNDERSPECIFIED` when an ambiguous target lacks a grounded repair.

Table 1: Controller semantics for DeGro integration.

Verifier state	Evidence	Controller action
Determinate	F sat, Φ unsat	answer with scope
Ambiguous	targets disagree	grounded repair
Inconsistent	F unsat	revise encoding
Unknown	solver inconclusive	fallback/escalate
Unsupported	outside fragment	alternate tool
Underspecified	no grounded repair	ask or abstain

These states have different controller implications. An unsupported construct should route to another tool or a broader compiler; an unknown query may warrant a larger solver budget; an inconsistent model requires revising the encoding; and an underspecified source should trigger clarification or abstention. Collapsing all four into “failure” would make both coverage reporting and agent behavior uninterpretable. Likewise, a determinate verdict authorizes only an answer scoped to F , not an assertion that the source has been fully formalized.

3.5 Agent Integration and Controller Semantics

DeGro is a verifier policy inside a larger agent. It converts an executable artifact into a typed observation that the controller maps to an action (Table 1).

In interactive use, `UNDERSPECIFIED` can request clarification; the benchmark scores abstention because no information channel exists. Separating generation, deterministic verification, and repair reduces self-confirmation and yields an audit trail. Interfaces should report that “the encoded constraints determine the target,” not that the source problem was formally proved.

4 Experiments

4.1 Setup

Compared systems and model settings. To answer RQ1, we compare `SELF-REVIEW`, `GROUNDING SELF-REVIEW`, `DETERMINACY-GUIDED REPAIR`, and `DeGro` under the same initial formalization and two-attempt budget. GPT-OSS 20B and 120B use medium reasoning, temperature 1.0, and frozen prompts; all $120 \times 2 \times 4 = 960$ records per model are present. The 120B substitution was selected after the original two-model plan and is therefore a post-hoc model comparison on the untouched fixed set. The paired construction tests source-sensitive behavior for RQ2, and a disjoint 100-pair exploratory study compares verdict-only and witness variants for RQ3.

Self-review receives the executable artifact and asks for correction without formal diagnosis. Grounded self-review adds the same source-citation requirement used by DeGro. Determinacy-guided repair supplies the typed solver verdict but no grounding interaction, isolating the diagnostic signal. DeGro combines that signal with source evidence and applies the same deterministic acceptance gate used in scoring. Since all methods begin from an identical F^- , differences cannot be attributed to initial-generation quality.

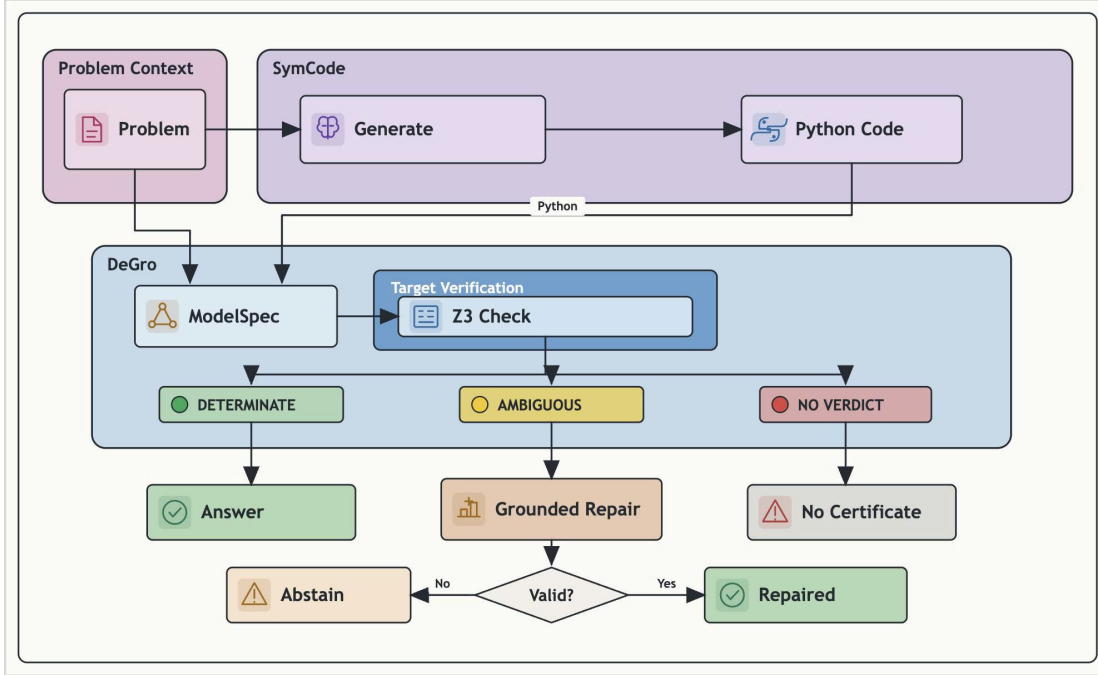


Figure 2: DeGro in the executable-reasoning pipeline. SymCode generates the executable artifact, while DeGro passes a structured MODELSPec to Z3-backed target-determinacy verification. A determinate target can be answered, an ambiguous target enters grounded repair and validation, and an inconclusive check returns no certificate.

4.2 Data

Controlled paired benchmark (RQ2). For each gold specification $F^* = \{c_1, \dots, c_k, c^*\}$, we form $F^- = F^* \setminus \{c^*\}$ and retain it only if F^* determines the target while F^- is satisfiable and target-ambiguous. Each pair shares F^- : an OMISSION source contains the fact for c^* and requires repair, whereas an UNDERSPECIFIED source omits it and requires abstention. The verifier state alone therefore cannot reveal the label. After excluding an 80-pair pilot and 100-pair development set, we froze 120 confirmatory MIRA-Math pairs (240 cases, five families) [1]; the combined 300-pair analysis is secondary.

Integrity checks. The builder imports the typed missing fact from MIRA-Math rather than asking an LLM to corrupt the problem, then applies all three solver checks. The final set exhausts compatible unseen identifiers; its manifest records source revision 72ff918, license, selection seed, family counts, and prompt/output hashes. The five families cover linear separation, rank-deficient systems, path sums, Chinese-remainder reconstruction, and moment constraints.

The families exercise distinct structures. Linear separation removes a relation that distinguishes otherwise compatible solutions; rank-deficient systems preserve several degrees of freedom. Graph path sums and Chinese-remainder reconstruction combine discrete equalities with domain restrictions, while moment problems couple polynomial summaries within the supported degree bound. Family-stratified results check that one easy template does not dominate the aggregate. Because every retained deletion is mechanically verified as target-critical, the benchmark does not mislabel an irrelevant omission as a repair opportunity.

Natural-error study. For ecological validity, GPT-OSS 20B generated one Structured SymCode specification per MATH-500 problem without injected errors. We froze all parsed, solver-supported, conclusive cases before annotation or outcome inspection and reused each initial specification across methods. Model-proposed labels and evidence were hidden from verifier outputs, then fully checked by two reviewers: all 292 MATH-500 and 503 ALG514 cases. Because separate pre-discussion human labels were not retained, we claim human verification, not independent annotation or inter-annotator agreement. Labels distinguish correct, missing/wrong/extra constraint, wrong domain, wrong target, computational error, and other; only source-supported target-critical omissions enter the primary subset.

For MATH-500, two reviewers inspected all 292 final labels from an archived model-assisted review. GPT-OSS 120B subsequently produced a fresh blinded proposal for every case under the same prompt; it agreed with the human-verified primary label on 207 of 292 cases (70.9%), illustrating why model proposals were not treated as ground truth. For ALG514, GPT-OSS 120B proposed 503 labels and re-adjudicated all 22 cases where semantic status disagreed with verifier-compatible numerical status; reviewers then inspected all final labels. Detector ambiguity could not define the positive class: a positive required a source-supported omission that can change the requested target.

For RQ3, primary natural-error measures are ambiguity recall and end-to-end grounded repair, with conclusive solver coverage reported over all inputs; secondary measures include precision, false ambiguity/repair, and preservation of correct controls. Proportions

use Wilson 95% intervals and paired answer accuracy uses exact McNemar. Numeric agreement alone is not repair success.

The original problem is the unit of analysis, and every method reuses its shared initial formalization. Recall asks how often DeGro identifies verified target-critical omissions; precision asks how often an ambiguous verdict corresponds to that class. Conditional repair counts accepted grounded fixes among detected omissions, whereas end-to-end repair retains every verified omission in the denominator. This distinction exposes the gap between finding ambiguity and recovering the missing source relation.

MATH-500 contributes 292 of 500 conclusive, solver-supported formalizations; ALG514 contributes 503 of 514. Exclusions are reported in the coverage results. DRAW-1K is omitted because corrected-verifier outcomes and final human labels were incomplete.

4.3 Main Experiment

Primary comparison. The preregistered 20B comparison tests whether DeGro improves FDA over grounded self-review. We apply the same exact paired McNemar test to the post-hoc 120B replacement and report Holm correction across the two model-size results. Paired percentage-point differences use pair-cluster bootstrap 95% intervals. Official model documentation describes configurable reasoning effort and structured output support for GPT-OSS [27].

Metrics and analysis. For RQ1, repair success rate (RSR) requires a grounded, functionally correct addition; correct abstention rate (CAR) rewards abstention on underspecified cases, and final decision accuracy (FDA) averages the balanced case types. Over-repair rate is $1 - \text{CAR}$, and unsupported repair (UR) is the share of additions lacking source support; UR is interpreted alongside RSR because abstaining on every case would trivially eliminate additions. Pair-cluster confidence intervals preserve within-pair dependence; semantic, functional, and grounded outcomes remain separate.

5 Results

RQ1: repair-or-abstain decisions. Table 2 reports the fixed 120-pair analysis. For GPT-OSS 20B, DeGro raises FDA from 90.0% to 95.0% relative to grounded self-review, a +5.0-point effect (95% CI [0.4, 9.6]; discordant counts 22 versus 10; exact McNemar $p = .050$). For GPT-OSS 120B, FDA rises from 87.5% to 93.8%, a +6.25-point effect (95% CI [1.7, 10.8]; 24 versus 9; $p = .0135$). Holm correction across these two model-specific comparisons gives $p = .0501$ and $p = .0271$, respectively. The 20B result follows the preregistered analysis; the 120B replacement is post-hoc but uses the same untouched cases, prompts, and scoring.

Table 2: Results on the fixed 120-pair set. The 20B comparison was preregistered; the 120B replacement is post-hoc. UR is unsupported additions divided by all proposed additions.

Model	Method	RSR↑	CAR↑	FDA↑	ORR↓	UR↓
GPT-OSS 20B	Self-review	93.3	87.5	90.4	12.5	15.2
	Grounded self-review	100.0	80.0	90.0	20.0	16.7
	Determinacy-guided	86.7	99.2	92.9	0.8	6.3
	DeGro	92.5	97.5	95.0	2.5	4.3
GPT-OSS 120B	Self-review	100.0	55.0	77.5	45.0	31.0
	Grounded self-review	95.8	79.2	87.5	20.8	17.9
	Determinacy-guided	98.3	84.2	91.2	15.8	15.1
	DeGro	97.5	90.0	93.8	10.0	10.7

RQ1: source-grounded acceptance. On the same confirmatory pairs, 20B DeGro lowers UR from 16.7% to 4.3% and raises CAR from 80.0% to 97.5%, while RSR falls from 100.0% to 92.5%. For 120B, DeGro lowers UR from 17.9% to 10.7%, raises CAR from 79.2% to 90.0%, and raises RSR from 95.8% to 97.5%. Thus the conservative action boundary improves for both sizes, although 20B trades some supported repairs for fewer unsupported additions.

6 Analysis

6.1 Cumulative and Natural-Error Analysis

Secondary cumulative analysis. Across all 300 pairs, DeGro versus grounded self-review changes FDA from 91.2% to 96.2% for 20B (difference +5.0 points, 95% CI [2.3, 7.8]; McNemar $p < .001$) and from 88.3% to 95.7% for 120B (difference +7.3 points, 95% CI [4.7, 10.0]; $p < .001$). These cumulative estimates include development data and do not replace the fixed 120-pair analysis.

RQ3: human-verified natural errors. On the supported MATH-500 cohort, 158 of 292 formalizations are correct; the remaining labels are 69 wrong-target, 32 wrong-constraint, and 24 target-critical missing-constraint, eight extra-constraint, and one wrong-domain. The detector identifies 21 of 24 target-critical omissions (87.5% recall, 95% Wilson CI [69.0, 95.7]) but precision is 17.4% (21/121), and only one omission is successfully repaired end to end (1/24 = 4.2%). DeGro nevertheless raises cohort answer accuracy from 49.3% (144/292) to 53.8% (157/292); 16 discordant cases favor DeGro and three favor the structured baseline (exact McNemar $p = .0044$). This prevalence is conditional on 58.4% solver-fragment coverage.

ALG514 [33] provides a higher-coverage contrast: 503 of 514 cases enter the verified cohort, of which 477 are correct, 11 wrong-target, eight missing-constraint, and seven wrong-constraint. The semantic-error rate is 5.2% (95% Wilson CI [3.6, 7.5]). Structured Solver and DeGro score 94.2% (484/514) and 94.4% (485/514), respectively, using order-invariant subset matching with 10^{-3} tolerance for ALG514’s all-unknown, rounded solution vectors. Nine numerically compatible answers retain a human-verified semantic error, directly showing that answer-only evaluation can conceal representation failures. The one-case DeGro gain on ALG514 is exploratory.

RQ3: detection–repair decomposition. The RQ3 results separate three stages that an aggregate answer score conflates. First, representation coverage asks whether the structured language and

solver can express and decide the generated model. Second, ambiguity detection asks whether target-divergent assignments exist within that model. Third, grounded repair asks whether the original source supports a new constraint that removes the disagreement. MATH-500 performs moderately on the first stage (292 of 500), strongly on recall at the second (21 of 24 omissions), and poorly on the third (one of 24). Thus RQ3 identifies conversion of a solver counterexample into a semantically correct, cited edit as the main bottleneck on this cohort.

Precision provides the complementary view. Only 21 of 121 ambiguous MATH-500 specifications are verified target-critical omissions. The remaining ambiguous cases include wrong targets, wrong constraints, and representations whose feasible set is broad for reasons other than a missing source fact. These are legitimate warnings about F but not opportunities for additive repair. A high-recall detector can consequently be useful for routing while still requiring a strict gate: automatically treating every ambiguity as an omission would reproduce the over-repair behavior observed in generic self-review.

Answer accuracy alone also understates the semantic distinction. On MATH-500, DeGro changes the shared structured baseline from 49.3% to 53.8%, with 16 discordances favoring DeGro and three favoring the baseline. Yet only one verified omission is repaired end to end, so most accuracy changes do not constitute evidence that the missing-relation mechanism was solved. On ALG514, nine numerically compatible outputs still carry a verified semantic defect. We therefore report numerical, semantic, functional, grounding, and determinacy outcomes separately rather than reinterpret every answer improvement as representation repair.

Decision-theoretic reading. FDA assigns equal mass to repair on complete sources and abstention on underspecified sources because every controlled item contributes one of each. Real applications may penalize fabrication more heavily than missed repair. The case-level release permits alternative utilities, but the qualitative tradeoff is already visible: generic grounded review attains high RSR while over-repairing, whereas a bare determinacy verdict favors abstention. DeGro attempts to recover useful repairs without surrendering the conservative action boundary. Whether that balance is preferable depends on the downstream cost of an unsupported assertion, which should be declared rather than hidden inside a single accuracy metric.

6.2 Witness Ablation and Family Effects

RQ3: witness ablation. On the exploratory 100-pair set, DeGro scores 98.5 FDA versus 97.5 with witnesses (five versus three discordances favoring DeGro; Holm-adjusted $p = .727$). A minimal witness raises RSR by six points over a bare verdict, also non-significantly ($p = .714$). Concrete assignments therefore add no measurable benefit here: the typed verdict can be exposed by default and witnesses retained for audit or harder cases.

This part of RQ3 is answered only on the exploratory controlled set. Every instance contains one target-critical omission, so non-uniqueness already localizes the relevant defect; witness assignments add tokens but little new information. Specifications with several missing facts or many nuisance dimensions may differ, but

Table 3: Exploratory GPT-OSS 20B ablation on 100 development pairs.

Method	RSR	CAR	FDA	ORR
Self-review	92	84	88.0	16
Bare non-uniqueness	87	97	92.0	3
Minimal witness	93	93	93.0	7
Target witnesses	86	94	90.0	6
Minimal witness+ground	97	99	98.0	1
DeGro	97	100	98.5	0
DeGro+witnesses	97	98	97.5	2

Table 4: Cumulative FDA by family for grounded self-review (GSR) and DeGro. Each family contains 60 pairs.

Family	GPT-OSS 20B		GPT-OSS 120B	
	GSR	DeGro	GSR	DeGro
CRT reconstruction	100.0	99.2	99.2	100.0
Graph path sums	96.7	99.2	88.3	97.5
Linear separation	76.7	97.5	85.0	90.8
Moment problem	88.3	97.5	79.2	95.0
Rank-deficient shared	94.2	87.5	90.0	95.0

these data support determinacy-guided grounding, not a universal benefit from exposing concrete assignments.

RQ1: unsupported additions and repair tradeoffs. For 20B, a bare verdict raises CAR to 98.0% while lowering RSR to 87.7%; DeGro restores repairs, reaches 99.0% CAR, and reduces unsupported additions from 12.5% to 3.4% relative to grounded self-review. For 120B, DeGro raises cumulative CAR from 79.7% to 93.3% and RSR from 97.0% to 98.0%, while reducing unsupported additions from 17.3% to 7.0%. Family effects remain concentrated rather than uniform.

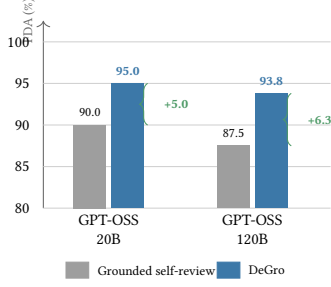
Table 4 explains the RQ1 tradeoff behind the aggregate scores. Generic review often inserts a plausible but unstated relation; DeGro blocks many such additions. The 20B rank-deficient family remains an adverse regime, whereas 120B improves in all five families. Lower UR therefore cannot by itself establish better grounded decision-making; RSR, CAR, and family-specific losses must be read with it.

6.3 Coverage Analysis

RQ3: broad-benchmark coverage audit. To measure the representation bottleneck preceding verification, GPT-OSS 20B generated one shared Structured SymCode specification for each MATH-500 problem [19]. Of 500 specifications, 292 (58.4%) parsed, reached the solver, and produced a conclusive verdict; Table 5 accounts for every exclusion.

Coverage is fixed solely from parsing and solver state, not observed answers. Solver-level unsupported cases dominate and require a broader theory or compiler; model-level failures instead require better structured generation. Conditional accuracy concerns the 292 supported problems, whereas end-to-end accuracy assigns failures to the other 208. All Structured Solver records completed; DeGro completed 285, with six call errors and one unsupported

(a) Confirmatory final decision accuracy



(b) MATH-500 diagnostic funnel

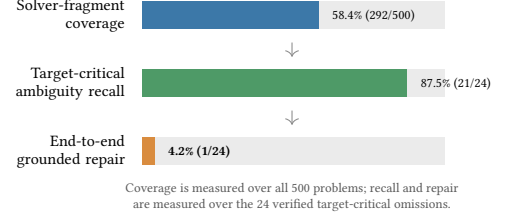


Figure 3: Two complementary views of the evidence. (a) DeGro improves FDA by 5.0 points for GPT-OSS 20B and 6.3 points for GPT-OSS 120B; the axis begins at 80% to make the paired differences visible. (b) Natural-error evaluation has a pronounced detection–repair gap: most target-critical omissions are detected, but only one is repaired end to end.

Table 5: Frozen MATH-500 cohort accounting for the shared initial structured specification.

Disposition	Count	Percent
Conclusive and solver-supported	292	58.4
Model-level unsupported	65	13.0
Solver-level unsupported	112	22.4
Inconsistent	9	1.8
Solver unknown	9	1.8
Call error	13	2.6
Total	500	100.0

record. We retain the frozen denominator and count all seven as incorrect.

This separation prevents a favorable post-hoc coverage slice. Conditional accuracy asks how well the agent performs once a specification reaches a conclusive solver state; end-to-end accuracy additionally charges every representation and execution failure. Reporting only the former would make a narrow verifier appear competitive with unrestricted program generation while hiding 41.6% of the benchmark. The exclusion categories also route improvements to the right component: unsupported solver constructs call for broader compilation, while failures before a valid MODEL-SPEC call for better structured generation.

6.4 Discussion and Limitations

Interpretation and validity. The paired design holds latent values, target, domains, retained constraints, and solver verdict fixed; only the source support for c^* changes. The repair policy never receives c^* , the gold target, or the gold action. The 120 confirmatory pairs were frozen before inspection, with tests and Holm correction declared in the manifest. The cumulative analysis remains secondary and the witness ablation exploratory. FDA equally weights repair and correct abstention; UR must be interpreted with RSR and CAR because always abstaining produces zero unsupported additions but no repair utility.

A conventional mixture of complete and incomplete questions admits shortcuts: families may differ in difficulty, wording may reveal that a fact is missing, and a policy may simply learn to repair

every ambiguous solver state. Our paired construction rules out these shortcuts because both members share the same formalization and diagnostic. Correct behavior changes only with the natural-language support for the withheld constraint. Exact McNemar tests then depend only on pairs for which the two policies disagree, while pair-cluster bootstrap intervals preserve the dependence between the two case types.

The primary contrast also controls the language-model interface. Both methods receive the same source, specification, output schema, grounding instruction, model, and token budget; the added determinacy signal is the intended treatment. Gold latent values are used only after generation for scoring. A direct target pin is considered unsupported unless a source span licenses it and all semantic, functional, and determinacy gates accept it. Retaining raw and gated outputs reveals when apparent accuracy is purchased by fabrication.

Scope of the claim. The evidence supports target determinacy as a guard against plausible but unstated repairs, including a five-point GPT-OSS gain. It does not establish semantic equivalence between P and F , a universal grounding benefit, an incremental witness benefit, or superiority over unrestricted code generation. Exact spans are auditable provenance rather than semantic entailment. Confidence and determinacy are complementary: low confidence may justify fallback for a determinate target, while confidence cannot authorize an ambiguous one.

The distinction is important for agent design. An execution monitor answers whether an artifact runs; an outcome verifier may check a final answer; DeGro instead characterizes the feasible set before the agent commits to a value. This intermediate certificate supports selective action: answer when the target is determined, repair only when a cited source fact restores determinacy, and otherwise ask, abstain, or escalate. It is not a proof that the formalization matches the source. A correctly solved but incorrectly translated model may remain determinate.

Grounding itself is also only a proxy. A model can cite the correct sentence yet encode the wrong equation, while an implicit mathematical convention may justify an expression without a contiguous span. We therefore keep semantic validity, functional validity,

provenance, and solver determinacy as separate fields. This decomposition makes disagreement auditable and prevents numerical answer correctness from silently absorbing translation errors.

External validity. MIRA cases are controlled but templated; MATH-500 is natural but only 58.4% lies inside the present solver fragment, whereas ALG514 reaches 97.9% coverage in a narrower domain. Labels were human-verified after model proposals, not independently annotated, and dataset prevalence is therefore reported separately. Natural errors contain only 24 MATH-500 and eight ALG514 target-critical omissions: detection is high but repair remains limited. The two GPT-OSS sizes also differ materially in baseline abstention behavior, bounding any size-general claim.

The controlled benchmark deliberately removes one known fact and therefore estimates a clean repair-versus-abstain decision under idealized representation control. It underrepresents paraphrase, irrelevant prose, diagrams, implicit conventions, wrong constraints, and interacting translation errors. The natural cohorts contain these phenomena but trade away causal control. Their purposes are complementary: the paired benchmark identifies a policy effect, while the natural study estimates what the current pipeline actually encounters. We do not merge them into one leaderboard number.

The annotation protocol also avoids defining positives with DeGro’s own verdict. Detector outputs, repairs, method names, and downstream answer outcomes are hidden from the initial packet. A target-critical omission must instead be established from the source and generated specification; correct, wrong-constraint, wrong-target, and wrong-domain cases remain in the denominator. Because model proposals seeded review and no separate blinded pre-discussion labels were preserved, we claim human verification rather than independent annotation or inter-annotator reliability.

Reproducibility and risk. The artifact includes the compiler, two-copy query, gates, tests, builders, frozen manifests, prompts, raw records, and analysis. The 20B and 120B confirmatory output hashes begin with 5616d0c6 and 38d18e69; each model analysis consumes all 960 expected records, and the supplement records the complete hashes. Determinacy remains relative to F : wrong or extra constraints may force a unique wrong target, non-target omissions are invisible, and the current SMT fragment excludes much nonlinear, transcendental, and diagrammatic reasoning. Interfaces must expose this scope rather than imply truth. The data contain no personal information; AI assistance requires venue-compliant disclosure and does not reduce author responsibility.

The manifests record dataset revision and license, selection seed, family counts, exact model strings, decoding settings, prompt hashes, timestamps, and full output hashes. Analysis scripts reconstruct every table from JSONL, reject incomplete expected pair counts, and preserve whether a row came from pilot, development, or confirmatory data. Natural-error packets and verified labels are separately blinded and content-addressed. These records allow the 120-pair headline to be reproduced without incorporating earlier observations.

Several failure modes remain outside the certificate. A wrong or extra constraint may force a unique but incorrect target; an omitted fact that does not affect q is invisible; several interacting omissions may exceed the bounded repair budget; and witnesses may differ only on irrelevant variables. The supported SMT fragment excludes

much nonlinear, transcendental, and diagrammatic mathematics. These are reasons to surface the encoded constraints, cited repair, and solver status in the interface, not to display an unqualified “verified” badge.

Finally, the two model sizes exhibit different instruction-following and abstention behavior, and hosted checkpoints may change. DeGro’s advantage should therefore be treated as a property of a specified model–prompt–gate configuration. The 120B baseline over-repairs many underspecified cases despite its larger size. Deterministic gates remain necessary, and deployment thresholds should reflect the relative cost of fabricated facts and missed repairs.

Deployment implications. DeGro is best used as a routing signal rather than a universal correctness badge. A determinate, supported specification can proceed to answer generation; an ambiguous specification with a grounded candidate can enter the bounded repair loop; and an ambiguous specification without support should trigger clarification or abstention. Inconsistent, unknown, and unsupported cases require different fallbacks, as Table 1 makes explicit. Keeping these states typed prevents an agent from treating lack of a solver certificate as permission to guess.

The results also suggest a conservative interface. The system should expose the target expression, the relevant encoded constraints, the determinacy status, and any source span used for repair. When a witness exists, it can be retained in the audit record even if the default language-model prompt receives only the verdict. This design follows the ablation result: witnesses are valuable mechanical evidence, but displaying them to the repair model did not improve decisions on the present benchmark. Users can then distinguish “the current model fixes this quantity” from the stronger and unsupported claim that the entire translation is correct.

Deployment comparisons should report representation coverage, decision quality conditional on a conclusive verifier state, and end-to-end utility with unsupported, unknown, and call-error cases retained as failures. Otherwise, a narrow symbolic verifier can appear reliable by declining difficult inputs before scoring.

7 Conclusion

For RQ1, DeGro raises FDA by 5.0 points for GPT-OSS 20B and 6.25 points for GPT-OSS 120B on the fixed 120-pair set while reducing unsupported additions for both sizes, though 20B sacrifices some supported repairs; the 120B substitution is post-hoc. For RQ2, the paired benchmark shows that correct behavior depends on source support even when the formalization and determinacy signal are identical. For RQ3, conclusive-verifier coverage and detection substantially exceed end-to-end grounded repair on natural errors, while concrete witnesses add no measurable benefit on the exploratory single-omission set. These answers support target determinacy as a representation-relative routing condition, with source grounding and coverage reported alongside decision accuracy.

References

- [1] Charbel Al Bateh and Samer Saab Jr. 2026. MIRA-Math: A Benchmark for Minimal Information Requesting and Mathematical Reasoning. *arXiv preprint arXiv:2607.07391* (2026).
- [2] Mohammadh Alhanahnah, Md Rashedul Hasan, and Hamid Bagheri. 2024. An Empirical Evaluation of Pre-trained Large Language Models for Repairing Declarative Formal Specifications. *arXiv preprint arXiv:2404.11050* (2024).
- [3] Paul C. Attie, Kinan Dak Al Bab, and Mouhammad Sakr. 2017. Model and Program Repair via SAT Solving. *ACM Transactions on Embedded Computing Systems* 17, 2, Article 32 (2017). doi:10.1145/3147426
- [4] Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf, Edward W. Ayers, Dragomir Radev, and Jeremy Avigad. 2023. ProofNet: Autoformalizing and Formally Proving Undergraduate-Level Mathematics. *arXiv preprint arXiv:2302.12433* (2023).
- [5] Sina Bagheri Nezhad, Yao Li, and Ameeta Agrawal. 2026. SymCode: A Neurosymbolic Approach to Mathematical Reasoning via Verifiable Code Generation. In *Findings of the Association for Computational Linguistics: EACL 2026*. Association for Computational Linguistics, 1489–1503. doi:10.18653/v1/2026.findings-eacl.76
- [6] Gilles Barthe, Juan Manuel Crespo, and César Kunz. 2011. Relational Verification Using Product Programs. In *FM 2011: Formal Methods (Lecture Notes in Computer Science, Vol. 6664)*. 200–214. doi:10.1007/978-3-642-21437-0_17
- [7] Gilles Barthe, Pedro R. D’Argenio, and Tamara Rezk. 2004. Secure Information Flow by Self-Composition. In *17th IEEE Computer Security Foundations Workshop*. 100–114. doi:10.1109/CSFW.2004.1310735
- [8] Wenhui Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. 2023. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *Transactions on Machine Learning Research* (2023).
- [9] Edmund M. Clarke, Orna Grumberg, Suresh Jha, Yuan Lu, and Helmut Veith. 2000. Counterexample-Guided Abstraction Refinement. In *Computer Aided Verification (Lecture Notes in Computer Science, Vol. 1855)*. 154–169. doi:10.1007/10722167_15
- [10] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. In *arXiv preprint arXiv:2110.14168*.
- [11] Wenyao Cui, Huaping Zhang, Yongyi Huang, Qiuchi Li, Jian Xu, Cheng-Lin Liu, Chunxiao Gao, Juan Wang, and Baohua Zhang. 2026. SymDiag: Explainable Diagnosis for LLM Reasoning via Neuro-Symbolic Verification. *arXiv preprint arXiv:2608.08786* (2026).
- [12] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems (Lecture Notes in Computer Science, Vol. 4963)*. Springer, 337–340. doi:10.1007/978-3-540-78800-3_24
- [13] Isil Dillig, Thomas Dillig, Boyang Li, and Kenneth L. McMillan. 2013. Inductive Invariant Generation via Abductive Inference. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 443–456. doi:10.1145/2491956.2462187
- [14] Ran El-Yaniv and Yair Wiener. 2010. On the Foundations of Noise-Free Selective Classification. *Journal of Machine Learning Research* 11 (2010), 1605–1641.
- [15] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: Program-aided Language Models. In *International Conference on Machine Learning*.
- [16] Yonatan Geifman and Ran El-Yaniv. 2017. Selective Classification for Deep Neural Networks. In *Advances in Neural Information Processing Systems*, Vol. 30.
- [17] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujia Yang, Nan Duan, and Weizhu Chen. 2024. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *International Conference on Learning Representations*.
- [18] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujia Yang, Minlie Huang, Nan Duan, and Weizhu Chen. 2024. ToRA: A Tool-Integrated Reasoning Agent for Mathematical Problem Solving. In *International Conference on Learning Representations*.
- [19] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, Vol. 1.
- [20] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large Language Models Cannot Self-Correct Reasoning Yet. In *International Conference on Learning Representations*.
- [21] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. 2023. Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. In *International Conference on Learning Representations*.
- [22] Junbo Jacob Lian, Yujun Sun, Huiling Chen, Chaoyu Zhang, Hanzhang Qin, and Chung-Piaw Teo. 2026. ReLoop: Structured Modeling and Behavioral Verification for Reliable LLM-Based Optimization. *arXiv preprint arXiv:2602.15983* (2026).
- [23] Mark H. Liffiton and Karem A. Sakallah. 2008. Algorithms for Computing Minimal Unsatisfiable Subsets of Constraints. *Journal of Automated Reasoning* 40 (2008), 1–33. doi:10.1007/s10817-007-9084-z
- [24] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let’s Verify Step by Step. *arXiv preprint arXiv:2305.20050* (2023).
- [25] Qing Lyu, Shreya Havaldar, Adam Stein, Li Zhang, Delip Rao, Eric Wong, Marianna Apidianaki, and Chris Callison-Burch. 2023. Faithful Chain-of-Thought Reasoning. In *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*. 305–329. doi:10.18653/v1/2023.ijcnlp-main.20
- [26] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36.
- [27] OpenAI. 2025. gpt-oss-120b & gpt-oss-20b Model Card. Accessed 2026-09-09. <https://openai.com/index/gpt-oss-model-card/>
- [28] Raymond Reiter. 1987. A Theory of Diagnosis from First Principles. *Artificial Intelligence* 32, 1 (1987), 57–95. doi:10.1016/0004-3702(87)90062-2
- [29] Dipankar Sarkar. 2026. From Errors to Proofs: Minimal-Core-Guided Repair for Neuro-Symbolic Constraint Solving. *arXiv preprint arXiv:2608.14771* (2026).
- [30] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit A. Seshia, and Vijay A. Saraswat. 2006. Combinatorial Sketching for Finite Programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*. 404–415. doi:10.1145/1168857.1168907
- [31] Tachio Terauchi and Alex Aiken. 2005. Secure Information Flow as a Safety Problem. In *Static Analysis (Lecture Notes in Computer Science, Vol. 3672)*. 352–367.
- [32] Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving Math Word Problems with Process- and Outcome-Based Feedback. *arXiv preprint arXiv:2211.14275* (2022).
- [33] Shyam Upadhyay and Ming-Wei Chang. 2017. Annotating Derivations: A New Evaluation Strategy and Dataset for Algebra Word Problems. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 1, Long Papers*. Association for Computational Linguistics, 494–504. <https://aclanthology.org/E17-1047/>
- [34] Ke Wang, Houxing Ren, Aojun Zhou, Zimu Lu, Sichun Luo, Weikang Shi, Renrui Zhang, Linqi Song, Mingjie Zhan, and Hongsheng Li. 2024. MathCoder: Seamless Code Integration in LLMs for Enhanced Mathematical Reasoning. In *International Conference on Learning Representations*.
- [35] Yuhuai Wu, Albert Qiaoju Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. 2022. Autoformalization with Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 35.